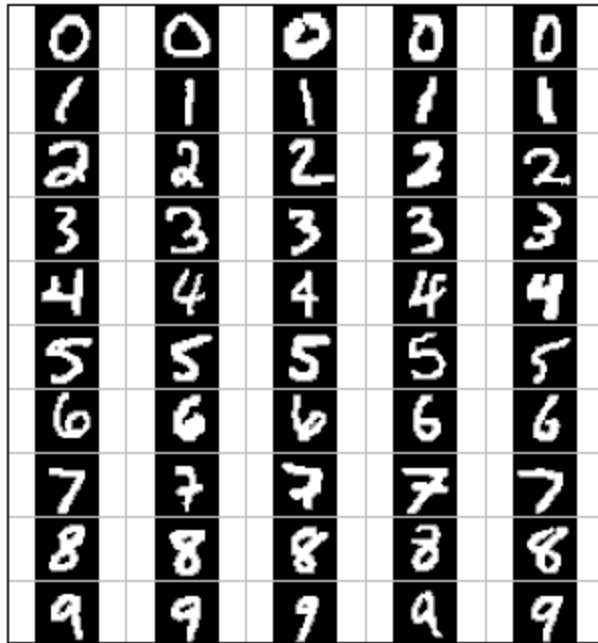


Acknowledgements

This project is part of the [Pac-man projects](#) created by [John DeNero](#) and [Dan Klein](#).

Optional Machine Learning Project - MIRA



Which Digit?



Which are Faces?

Introduction

In this project, you will design a large-margin (MIRA) classifier. You will test your classifiers on two image data sets: a set of scanned handwritten digit images and a set of face images in which edges have already been detected. Even with simple features, your classifiers will be able to do quite well on these tasks when given enough training data.

Optical character recognition (**OCR**) is the task of extracting text from image sources. The first data set on which you will run your classifiers is a collection of handwritten numerical digits (0-9). This is a very commercially useful technology, similar to the technique used by the US post office to route mail by zip codes. There are systems that can perform with over 99% classification accuracy.

Face detection is the task of localizing faces within video or still images. The faces can be at any location and vary in size. There are many applications for face detection, including human computer interaction and surveillance. You will attempt a simplified face detection task in which your system is presented with an image that has been pre-processed by an edge detection algorithm. The task is to determine whether the edge image is a face or not. There are several systems in use that perform quite well at the face detection task.

The code for this project includes the following files and data, available as a [zip file](#) (please note that this has code for other classifiers that you may ignore).

Data file

[data.zip](#) Data file, including the digit and face data.

Files you will edit

[mira.py](#) The location where you will write your MIRA classifier.

[dataClassifier.py](#) The wrapper code that will call your classifiers. You will use this code to analyze the behavior of your classifier and may edit some parts of it if you want to.

[answers.py](#) Answers to Question 2 and Question 4 go here.

Files you should read but NOT edit

[classificationMethod.py](#) Abstract super class for the classifiers you will write.
(You **should** read this file carefully to see how the infrastructure is set up.)

[samples.py](#) I/O code to read in the classification data.

`util.py`

Code defining some useful tools. You may be familiar with some of these by now, and they will save you a lot of time.

`mostFrequent.py`

A simple baseline classifier that just labels every instance as the most frequent class.

`runMinicontest.py`

The command you will use to run the minicontest, if you decide to enter.

What to submit: You will fill in portions of `answers.py`, `mira.py` and `dataClassifier.py` (only) during the assignment, and submit them.

Evaluation: Your code will be autograded for technical correctness. Please *do not* change the names of any provided functions or classes within the code, or you will wreak havoc on the autograder.

Getting Started

To try out the classification pipeline, run `dataClassifier.py` from the command line. This will classify the digit data using the default classifier (`mostFrequent`) which blindly classifies every example with the most frequent label.

```
python dataClassifier.py
```

As usual, you can learn more about the possible command line options by running:

```
python dataClassifier.py -h
```

We have defined some simple features for you. Later you will design some better features. Our simple feature set includes one feature for each pixel location, which can take values 0 or 1 (off or on). The features are encoded as a `Counter` where keys are feature locations (represented as (column,row)) and values are 0 or 1. The face recognition data set has value 1 only for those pixels identified by a Canny edge detector.

Implementation Note: You'll find it easiest to hard-code the binary feature assumption. If you do, make sure you don't include any non-binary features. Or, you can write your code more generally, to handle arbitrary feature values, though this will probably involve a preliminary pass through the training set to find all possible feature values (and

you'll need an "unknown" option in case you encounter a value in the test data you never saw during training).

MIRA

A skeleton implementation of the MIRA classifier is provided for you in [mira.py](#). MIRA is an online learner which is closely related to both the support vector machine and perceptron classifiers. You will fill in the `trainAndTune` function.

Theory

Similar to a multi-class perceptron classifier, multi-class MIRA classifier also keeps a weight vector w^y of each label y . We also scan over the data, one instance at a time. When we come to an instance (f, y) , we find the label with highest score:

$$y' = \arg \max_{y''} \text{score}(f, y'')$$

We compare y' to the true label y . If $y' = y$, we've gotten the instance correct, and we do nothing. Otherwise, we guessed y' but we should have guessed y . Unlike perceptron, we update the weight vectors of these labels with variable step size:

$$\begin{aligned} w^y &= w^y + \tau f \\ w^{y'} &= w^{y'} - \tau f \end{aligned}$$

where $\tau \geq 0$ is chosen such that it minimizes

$$\min_{w'} \frac{1}{2} \sum_c ||(w')^c - w^c||_2^2$$

subject to the condition that $(w')^y f \geq (w')^{y'} f + 1$

which is equivalent to

$$\min_{\tau} ||\tau f||_2^2 \text{ subject to } \tau \geq \frac{(w^{y'} - w^y)f + 1}{2||f||_2^2} \text{ and } \tau \geq 0$$

Note that, $w^{y'} f \geq w^y f$, so the condition $\tau \geq 0$ is always true given

$$\tau \geq \frac{(w^{y'} - w^y)f + 1}{2||f||_2^2} \text{ Solving this simple problem, we then have}$$

$$\tau = \frac{(w^{y'} - w^y)f + 1}{2||f||_2^2}$$

However, we would like to cap the maximum possible value of τ by a positive constant C , which leads us to

$$\tau = \min \left\{ C, \frac{(w^{y'} - w^y)f + 1}{2\|f\|_2^2} \right\}$$

Question 5 (6 points) Implement `trainAndTune` in `mira.py`. This method should train a MIRA classifier using each value of C in `Cgrid`. Evaluate accuracy on the held-out validation set for each C and choose the C with the highest validation accuracy. In case of ties, prefer the *lowest* value of C . Test your MIRA implementation with:

```
python dataClassifier.py -c mira --autotune
```

Hints and observations:

- Pass through the data `self.max_iterations` times during training.
- Store the weights learned using the best value of C at the end in `self.weights`, so that these weights can be used to test your classifier.
- To use a fixed value of $C=0.001$, remove the `--autotune` option from the command above.
- Validation and test accuracy when using `--autotune` should be in the 60's.
- It might save some debugging time if the $+1$ term above is implemented as $+1.0$, due to division truncation of integer arguments. Depending on how you implement this, it may not matter.
- The same code for returning high odds features in your perceptron implementation should also work for MIRA if you're curious what your classifier is learning.

To run the autograder for this question and visualize the output:

```
python autograder.py -q q5
```